

```
import pandas as pd

sales_data = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct"],
    "Units_Sold": [120, 135, 150, 160, 175, 180, 195, 210, 220, 240],
    "Price_Per_Unit": [50, 50, 55, 55, 60, 60, 65, 65, 70, 70],
    "Customer_Count": [80, 90, 100, 110, 120, 130, 140, 150, 160, 170],
    "Discount": [5, 5, 10, 10, 15, 15, 20, 20, 25, 25],
    "Sales_Amount": [6000, 6750, 8250, 8800, 10500, 10800, 12675, 13650, 15400, 16800]
})

print(sales_data)
```

	Month	Units_Sold	Price_Per_Unit	Customer_Count	Discount	Sales_Amount
0	Jan	120	50	80	5	6000
1	Feb	135	50	90	5	6750
2	Mar	150	55	100	10	8250
3	Apr	160	55	110	10	8800
4	May	175	60	120	15	10500
5	Jun	180	60	130	15	10800
6	Jul	195	65	140	20	12675
7	Aug	210	65	150	20	13650
8	Sep	220	70	160	25	15400
9	Oct	240	70	170	25	16800

```
print(sales_data.info())

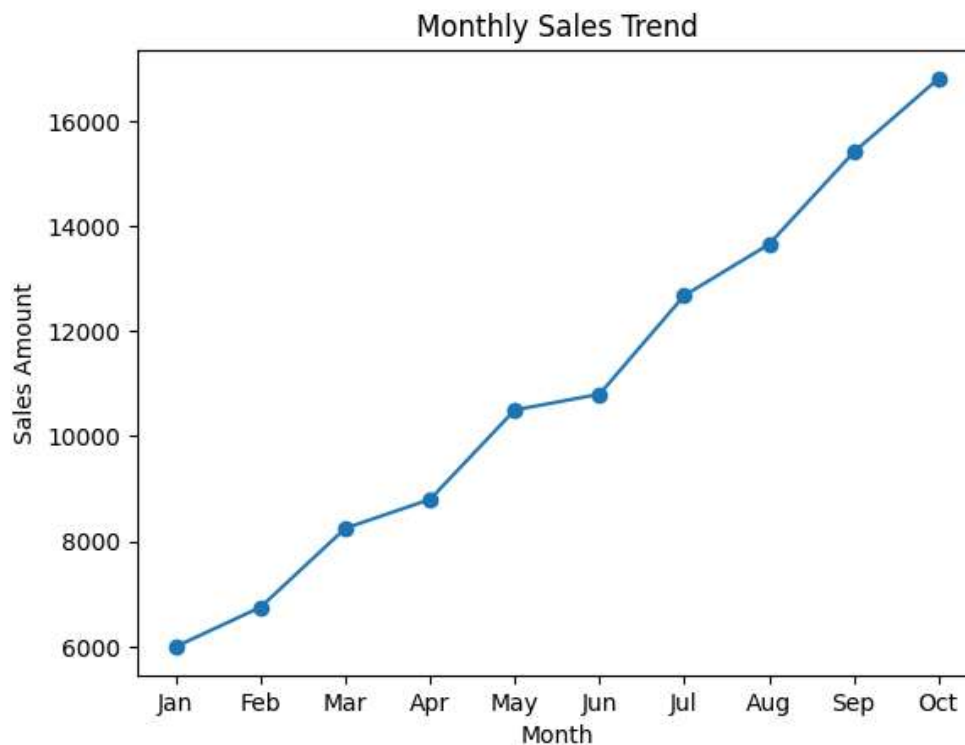
print(sales_data.isnull().sum())

print(sales_data.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Month           10 non-null    object
1   Units_Sold      10 non-null    int64
2   Price_Per_Unit  10 non-null    int64
3   Customer_Count  10 non-null    int64
4   Discount        10 non-null    int64
5   Sales_Amount    10 non-null    int64
dtypes: int64(5), object(1)
memory usage: 612.0+ bytes
None
Month           0
Units_Sold      0
Price_Per_Unit  0
Customer_Count  0
Discount        0
Sales_Amount    0
dtype: int64
Units_Sold  Price_Per_Unit  Customer_Count  Discount  Sales_Amount
count      10.000000      10.000000      10.000000    10.000000    10.000000
mean       178.500000      60.000000     125.000000    15.000000   10962.500000
std         38.373168       7.45356      30.276504     7.45356     3629.647404
min        120.000000     50.000000     80.000000     5.000000    6000.000000
25%        152.500000     55.000000    102.500000    10.000000   8387.500000
50%        177.500000     60.000000    125.000000    15.000000   10650.000000
75%        206.250000     65.000000    147.500000    20.000000   13406.250000
max        240.000000     70.000000    170.000000    25.000000   16800.000000
```

```
import matplotlib.pyplot as plt

plt.plot(sales_data["Month"], sales_data["Sales_Amount"], marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

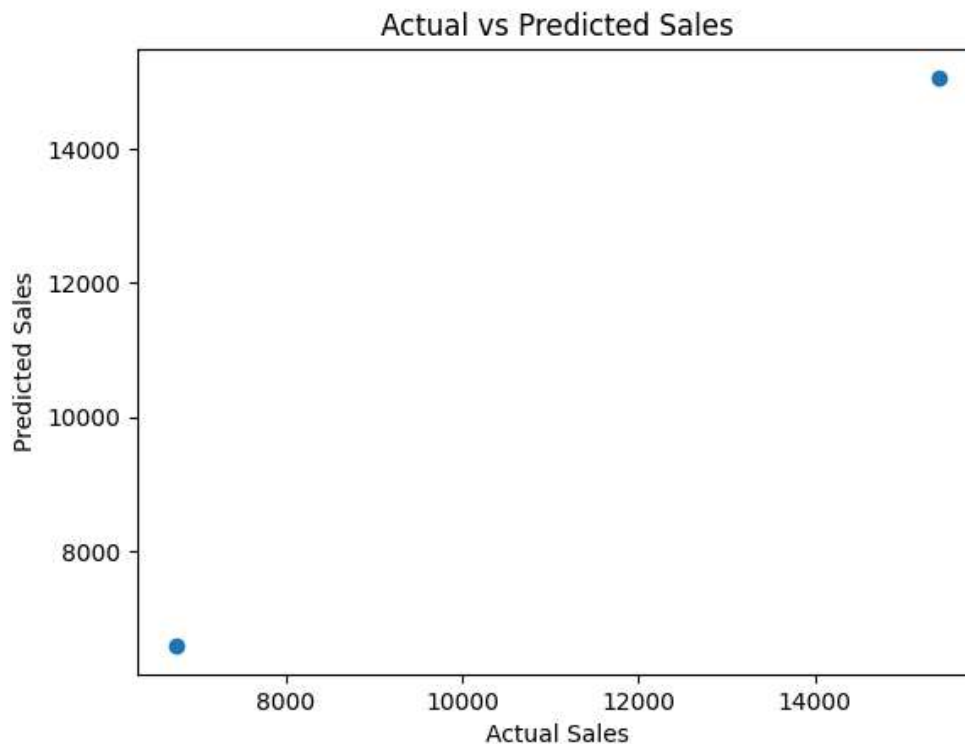
X = sales_data[["Units_Sold", "Price_Per_Unit", "Customer_Count", "Discount"]]
y = sales_data["Sales_Amount"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")
plt.show()
```



```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
# Evaluation Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)
```

```
Mean Absolute Error (MAE): 248.79807692307259
Mean Squared Error (MSE): 69472.82960428741
Root Mean Squared Error (RMSE): 263.5769898991325
R2 Score: 0.9962859926035998
```

Start coding or [generate](#) with AI.

